M2PGi – Internet Of Things

Pr. Olivier Gruber

(olivier.gruber@univ-grenoble-alpes.fr)

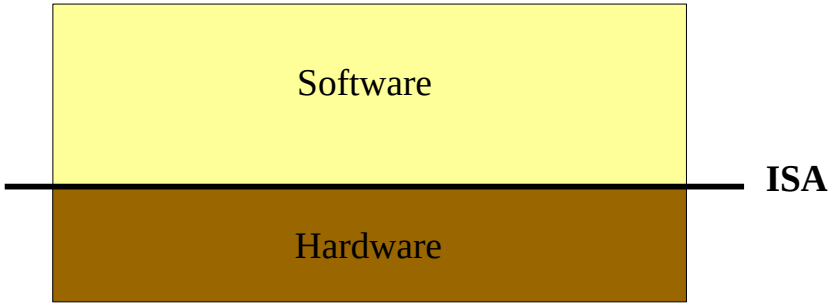
Laboratoire d'Informatique de Grenoble

Université de Grenoble-Alpes

© Pr. Olivier Gruber

Today

- Analyses of our code
- Designing solutions



© Pr. Olivier Gruber

Analysis – Top-Level Loop

How do these codes work?
- dialog with peripherals
- ***blocking or non-blocking calls***

How well do these codes work?
- power consumption
- correctness

Could we mix them?

```
void _start() {

  uart_init(UART0);
  for (;;) {
    // read available byte from the keyboard
    uint8_t code = uart_receive(UART0);

    // echo the character to the terminal
    uart_send(UART0, code);
  }
}
```

```
uint8_t button_get_status(void* bar);
void led_on(void* bar);
void led_off(void* bar);


void start() {

  button_init_regs(BUTTON_BAR);
  led_init_regs(LED_BAR);
  for (;;) {
     uint8_t status = button_get_status(BUTTON_BAR);
     if (status & 0x01)
       led_on(LED_BAR);
      else
       led_off(LED_BAR);
  }
}
```

© Pr. Olivier Gruber

Analysis – Button & LED Example


```
uint8_t button_get_status(void* bar);
void led_on(void* bar);
void led_off(void* bar);


void start() {

  button_init_regs(BUTTON_BAR);
  led_init_regs(LED_BAR);
  for (;;) {
     uint8_t status = button_get_status(BUTTON_BAR);
     if (status & 0x01)
       led_on(LED_BAR);
      else
       led_off(LED_BAR);
  }
}
```

Does it work? Sure.
But how well?

© Pr. Olivier Gruber

Analysis – Button & LED Example


```
uint8_t button_get_status(void* bar);
void led_on(void* bar);
void led_off(void* bar);


void start() {

  button_init_regs(BUTTON_BAR);
  led_init_regs(LED_BAR);
  for (;;) {
     uint8_t status = button_get_status(BUTTON_BAR);
     if (status & 0x01)
       led_on(LED_BAR);
      else
       led_off(LED_BAR);
  }
}
```

Problem:
continuous polling/spinning.

→ processor never halts
→ high power consumption

Limitation:
our software does only one thing/task, what if we had multiple tasks to carry out?

© Pr. Olivier Gruber

Analysis – The Console


```
void _start() {

  uart_init(UART0);

  for (;;) {
    // read available byte from the keyboard
    uint8_t code = uart_receive(UART0);

    // echo the character to the terminal
    uart_send(UART0, code);

  }
}
```

How is this code working?

Is it spinning or not?

In other words, is the function ***uart_receive*** ***blocking*** or ***non-blocking***?

© Pr. Olivier Gruber

Analysis – UART Code


```c
/*
 * Reads one byte, if available, otherwise spin-waits.
 */
uint8_t uart_read(void* uart){
  while ((mmio_read32(uart,UART_SR) & UART_SR_REMPTY))
    ;
  return mmio_read32(uart,UART_FIFO);
}
```

Looks like blocking

```c
/*
 * Writes one byte, if there is room to do so, otherwise spin-waits.
 */
void uart_write(void* uart, uint8_t bits){
  while ((mmio_read32(uart,UART_SR) & UART_SR_TFUL))
    ;
  mmio_write32(uart,UART_FIFO, bits);
}
```

Looks like blocking

© Pr. Olivier Gruber

Analysis – UART Code


```
/*
 * Non-blocking read from the given UART
 */
uint8_t uart_read(void* uart){
  if ((mmio_read32(uart,UART_SR) & UART_SR_REMPTY))
    return  0;
  return mmio_read32(uart,UART_FIFO);
}
```

non-blocking version

```
/*
 * Blocking write to the given uart,
 * if there is room to do so, otherwise spin-waits.
 */
void uart_write(void* uart, uint8_t bits){
  while ((mmio_read32(uart,UART_SR) & UART_SR_TFUL))
    ;
  mmio_write32(uart,UART_FIFO, bits);
}
```

still blocking!

A non-blocking would be really hard to use.

© Pr. Olivier Gruber

Analysis – The Console


```
void _start() {

  uart_init(UART0);

  for (;;) {
    // read available byte from the keyboard
    uint8_t code = uart_receive(UART0);

    if (code==0) continue;

    // echo the character to the terminal
    uart_send(UART0, code);

  }
}
```

A non-blocking version of our code

See, it is important to know
if the functions called by your code
are ***blocking*** or ***non-blocking***...

© Pr. Olivier Gruber

Analysis – The Console


```
void _start() {

  uart_init(UART0);

  for (;;) {
    // read available byte from the keyboard
    uint8_t code = uart_receive(UART0);

    if (code==0) continue;

    // echo the character to the terminal
    uart_send(UART0, code);

  }
}
```

How well does this code work?

© Pr. Olivier Gruber

Analysis – The Console


```
void _start() {

  uart_init(UART0);

  for (;;) {
    // read available byte from the keyboard
    uint8_t code = uart_receive(UART0);

    if (code==0) continue;

    // echo the character to the terminal
    uart_send(UART0, code);

  }
}
```

Not so well...

It is ***still spinning***, *most of the time*, waiting for bytes to be received by the UART...

Note: if the send blocks...
the code no longer reads...

© Pr. Olivier Gruber

Analysis – The Console


```
void shell(char* line, int length);

char line[80];
int offset = 0;

void _start() {

  uart_init(UART0);

  for (;;) {
    // read available byte from the keyboard
    uint8_t code = uart_receive(UART0);

    if (code==0) continue;

    // echo the character to the terminal
    uart_send(UART0, code);
    if (code == '\n') {
      shell(line, offset);
      offset=0;
    } else
      line[offset++]=(char)code;
  }
}
```

Adding a shell that interprets command lines entered at the console...

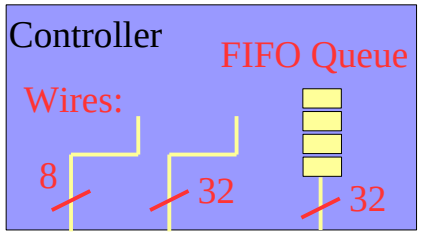
Executing commands take longer than others.

Problems? Think hardware limits...

© Pr. Olivier Gruber

Analysis – The Console


```
void shell(char* line, int length);

char line[80];
int offset = 0;

void _start() {

  uart_init(UART0);

  for (;;) {
    // read available byte from the keyboard
    uint8_t code = uart_receive(UART0);

    if (code==0) continue;

    // echo the character to the terminal
    uart_send(UART0, code);
    if (code == '\n') {
      shell(line, offset);
      offset=0;
    } else
      line[offset++]=(char)code;
  }
}
```



UART with 8-byte FIFO
115200 bps → over 14000 Bps
One byte every 70 us

*Dropping bytes after **560 us***
560 us = (8 * 70us)

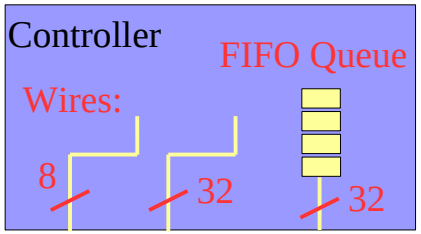
Embedded ARM Core @ 30MHz
1 cycle = 1 instruction
560 us = 560*30 = 16800 instructions

Does this code drop received bytes?

© Pr. Olivier Gruber

Analysis – The Console


```
void shell(char* line, int length);

char line[80];
int offset = 0;

void _start() {

  uart_init(UART0);

  for (;;) {
    // read available byte from the keyboard
    uint8_t code = uart_receive(UART0);

    if (code==0) continue;

    // echo the character to the terminal
    uart_send(UART0, code);
    if (code == '\n') {
      shell(line, offset);
      offset=0;
    } else
      line[offset++]=(char)code;
  }
}
```



Analysis:

1. If the end user waits the end of the previous command to type some more characters, this code works just fine.

2. If the end user is an expert, typing away several commands in a row, blind (no echo), this code will fail.

Nota Bene: timing matters...
this code may drop received bytes!

© Pr. Olivier Gruber

Analysis – UART Code – **???**

How do we dialog with hardware peripherals/devices?

```
uint8_t uart_read(void* uart){
  if ((mmio_read32(uart,UART_SR) & UART_SR_REMPTY))
    return  0;
  return mmio_read32(uart,UART_FIFO);
}
```

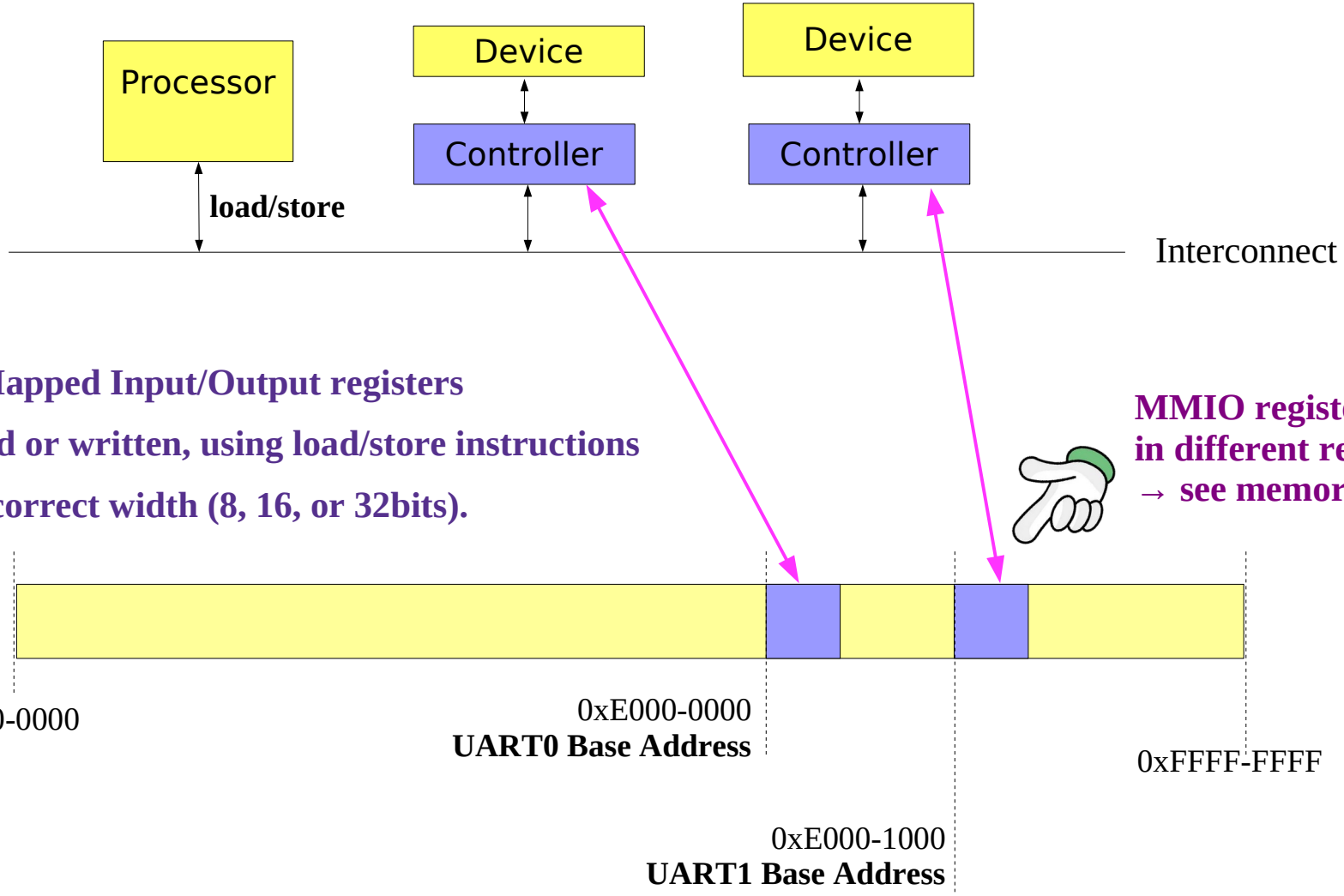
mmio ???

```
void uart_write(void* uart, uint8_t bits){
  while ((mmio_read32(uart,UART_SR) & UART_SR_TFUL))
    ;
  mmio_write32(uart,UART_FIFO, bits);
}
```

mmio ???

© Pr. Olivier Gruber

Memory-Mapped Input/Output Registers (MMIO)



Memory-Mapped Input/Output registers

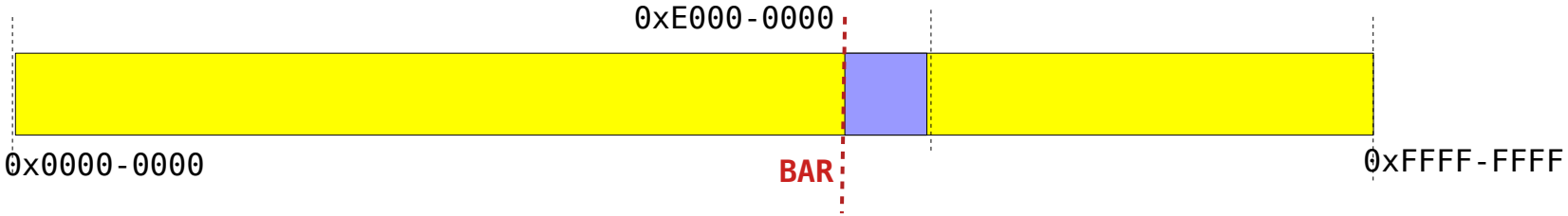
Can be read or written, using load/store instructions

But of the correct width (8, 16, or 32bits).

MMIO registers in different regions → see memory map

© Pr. Olivier Gruber

MMIO registers – Take Width Into Account




```
__inline__
__attribute__((always_inline))
uint16_t mmio_read16(void* bar, uint32_t offset) {
  return *((uint16_t*)(bar+offset));
}

__inline__
__attribute__((always_inline))
void mmio_write16(void* bar, uint32_t offset, uint16_t value) {
  *((uint16_t*)(bar+offset)) = value;
}

__inline__
__attribute__((always_inline))
void mmio_set16bits(void* bar, uint32_t offset, uint16_t bits) {
  uint16_t value = *((uint16_t*)(bar+offset));
  value |= bits;
  *((uint16_t*)(bar+offset)) = value;
}

__inline__
__attribute__((always_inline))
void mmio_clear16bits(void* bar, uint32_t offset, uint16_t bits) {
  uint16_t value = *((uint16_t*)(bar+offset));
  value &= ~bits;
  *((uint16_t*)(bar+offset)) = value;
}
```

Read and write mmio registers that are ***16-bit wide***

Note: always inlined

Manipulating bits of mmio registers that are ***16-bit wide.***

Note: always inlined

© Pr. Olivier Gruber

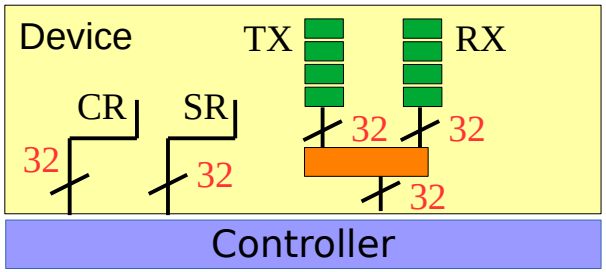
UART Device – MMIO Registers

From the Zynq-7000/R1P8 Technical Reference Manual

- Control Register (32-bits ***CR***)
- Status Register (32-bits ***SR***)
- Transmit (***TX***) & Receive (***RX***) FIFO

relevant mmio registers

FIFO depths: 8 registers



FIFO Queues
TX RX

```
#define UART_CR          0x0000 /* UART Control Register */
#define UART_MR          0x0004 /* UART Mode Register */
#define UART_IER         0x0008 /* -- Interrupt Enable Register */
#define UART_IDR         0x000C /* -- Interrupt Disable Register */
#define UART_IMR         0x0010 /* -- Interrupt Mask Register */
#define UART_ISR         0x0014 /* -- Channel Interrupt Status Register */
#define UART_BAUDGEN     0x0018 /* Baude Rate Generator Register */
#define UART_RXTOUT      0x001C /* -- Receiver Timeout Register */
#define UART_RXWM        0x0020 /* -- Receiver FIFO Trigger Level Register */
#define UART_MODEMCR     0x0024 /* -- Modem Control Register */
#define UART_MODEMSR     0x0028 /* -- Modem Status Register */
#define UART_SR          0x002C /* Channel Status Register */
#define UART_FIFO        0x0030 /* Transmit & Receive FIFO */
#define UART_BAUDDIV     0x0034 /* Baud Rate Divider Register */
#define UART_FLOWD       0x0038 /* -- Flow Control Delay Register */
#define UART_TXWM        0x0044 /* -- Transmitter FIFO Trigger Level Register */
```

Offsets from the **Base Address**

UART0 Base Address: 0xE000-0000
UART1 Base Address: 0xE000-1000

© Pr. Olivier Gruber

UART Device – Reading/Receiving (RX)


```
#define UART_SR          0x002C /* Channel Status Register */
#define UART_FIFO        0x0030 /* Transmit & Receive FIFO */
/*
 * Channel Status Register (UART_SR)
 * Bit-field masks:
 */
#define UART_SR_TNFUL    (1 << 14)
#define UART_SR_TTRIG    (1 << 13)
#define UART_SR_FDELT    (1 << 12)
#define UART_SR_TACTIVE  (1 << 11)
#define UART_SR_RACTIVE  (1 << 10)
#define UART_SR_TFUL     (1 <<  4)
#define UART_SR_TEMPTY   (1 <<  3)
#define UART_SR_RFUL     (1 <<  2)
#define UART_SR_REMPTY   (1 <<  1)    // Mask to know if the RX FIFO is empty.
#define UART_SR_RTRIG    (1 <<  0)
```

Masks defined based on the bit-field definition in the documentation.

```
uint8_t uart_read(void* uart){
  if ((mmio_read32(uart,UART_SR) & UART_SR_REMPTY))
    return  0;
  return mmio_read32(uart,UART_FIFO);
}
```

© Pr. Olivier Gruber

UART Device – Writing/Transmitting (TX)


```
#define UART_SR          0x002C /* Channel Status Register */
#define UART_FIFO        0x0030 /* Transmit & Receive FIFO */
/*
 * Channel Status Register (UART_SR)
 * Bit-field masks:
 */
#define UART_SR_TNFUL    (1 << 14)
#define UART_SR_TTRIG    (1 << 13)
#define UART_SR_FDELT    (1 << 12)
#define UART_SR_TACTIVE  (1 << 11)
#define UART_SR_RACTIVE  (1 << 10)
#define UART_SR_TFUL     (1 << 4)    // Mask to know if the TX FIFO is full.
#define UART_SR_TEMPTY   (1 << 3)
#define UART_SR_RFUL     (1 << 2)
#define UART_SR_REMPTY   (1 << 1)
#define UART_SR_RTRIG    (1 << 0)
```

```
void uart_write(void* uart, uint8_t bits){
  while ((mmio_read32(uart,UART_SR) & UART_SR_TFUL))
    ;
  mmio_write32(uart,UART_FIFO, bits);
}
```

© Pr. Olivier Gruber

Analysis – Back to our Top-Level Loop


```
void _start() {

  uart_init(UART0);
  for (;;) {
    // read available byte from the keyboard
    uint8_t code = uart_receive(UART0);

    // echo the character to the terminal
    uart_send(UART0, code);
  }
}
```

How do these codes work? ✓

- dialog with peripherals
- ***blocking or non-blocking calls***

How well do these codes work? ✓

- power consumption
- correctness

Could we mix them?

```
uint8_t button_get_status(void* bar);
void led_on(void* bar);
void led_off(void* bar);


void start() {

  button_init_regs(BUTTON_BAR);
  led_init_regs(LED_BAR);
  for (;;) {
     uint8_t status = button_get_status(BUTTON_BAR);
     if (status & 0x01)
       led_on(LED_BAR);
      else
       led_off(LED_BAR);
  }
}
```

© Pr. Olivier Gruber

Analysis – Top-Level Loop – Mixing Tasks


```
void _start() {

  uart_init(UART0);
  for (;;) {
    // read available byte from the keyboard
    uint8_t code = uart_receive(UART0);

    // echo the character to the terminal
    uart_send(UART0, code);
  }
}
```

Not with the blocking version!

How do these codes work? ✓

- dialog with peripherals
- ***blocking or non-blocking calls***

How well do these codes work? ✓

- power consumption
- correctness

Could we mix them?

```
uint8_t button_get_status(void* bar);
void led_on(void* bar);
void led_off(void* bar);


void start() {

  button_init_regs(BUTTON_BAR);
  led_init_regs(LED_BAR);
  for (;;) {
     uint8_t status = button_get_status(BUTTON_BAR);
     if (status & 0x01)
       led_on(LED_BAR);
      else
       led_off(LED_BAR);
  }
}
```

© Pr. Olivier Gruber

Analysis – Top-Level Loop – Mixing Non-blocking Tasks


```
void _start() {
  uart_init(UART0);
  for (;;) {
    // read available byte from the keyboard
    uint8_t code = uart_receive(UART0);
    if (code==0) continue;
    // echo the character to the terminal
    uart_send(UART0, code);
  }
}
```

Non-blocking version!

Could we mix them now?

```
uint8_t button_get_status(void* bar);
void led_on(void* bar);
void led_off(void* bar);


void start() {
  button_init_regs(BUTTON_BAR);
  led_init_regs(LED_BAR);
  for (;;) {
     uint8_t status = button_get_status(BUTTON_BAR);
     if (status & 0x01)
       led_on(LED_BAR);
      else
       led_off(LED_BAR);
  }
}
```

© Pr. Olivier Gruber

Example – An Actual Mix of Tasks

Here is a mix of the two...

```
void _start() {
  uart_init(UART0);
  button_init_regs(BUTTON_BAR);
  led_init_regs(LED_BAR);

  for (;;) {

    uint8_t status = button_get_status(BUTTON_BAR);
    if (status & 0x01)
      led_on(LED_BAR);
    else
      led_off(LED_BAR);

    // read available byte from the keyboard
    uint8_t code = uart_receive(UART0);
    if (code==0) continue;
    // echo the character to the terminal
    uart_send(UART0, code);
  }
}
```

Well, it would work...

But how messy is this loop going to get as we have more and more "tasks" to do...

© Pr. Olivier Gruber

Cleaned-up Example – An Actual Mix of Tasks


```
void _start() {
  uart_init(UART0);
  button_init_regs(BUTTON_BAR);
  led_init_regs(LED_BAR);

  for (;;) {
    button_and_led();
    echo();
  }
}

void button_and_led() {
  uint8_t status = button_get_status(BUTTON_BAR);
  if (status & 0x01)
    led_on(LED_BAR);
  else
    led_off(LED_BAR);
}

void echo() {
  // read available byte from the keyboard
  uint8_t code = uart_receive(UART0);
  if (code==0) return;
  // echo the character to the terminal
  uart_send(UART0, code);
}
```

Of course, we can do that...

OK, but still...

We are polling the button far too often, because we need to poll the UART at a high frequency (up to 115200Hz)...

We would like to process each function at a different period, **the right period for each task**.

© Pr. Olivier Gruber

Example – A More Realistic Mix of Tasks

Here is a more realistic mix with
- button & led
- a console with a ***shell***...

What about now?

How is this mix working?

```
void _start() {
  uart_init(UART0);
  button_init_regs(BUTTON_BAR);
  led_init_regs(LED_BAR);
  for (;;) {
    button_and_led();
    echo();
  }
}
void button_and_led() {
  uint8_t status = button_get_status(BUTTON_BAR);
  if (status & 0x01)
    led_on(LED_BAR);
  else
    led_off(LED_BAR);
}
void echo() {
  // read available byte from the keyboard
  uint8_t code = uart_receive(UART0);
  if (code==0) return;
  // echo the character to the terminal
  uart_send(UART0, code);
  if (code == '\n') {
      shell(line, offset);
      offset=0;
    } else
      line[offset++]=(char)code;
}
```

© Pr. Olivier Gruber

Analysis – A More Realistic Mix of Tasks

Here is a more realistic mix with
- button & led
- a console with a ***shell***...

Analysis

- If the shell takes a long time for certain commands, the ***button may feel unresponsive***...

- If we would have another UART to read from... ***we could certainly drop bytes***...

So we need that ***each function***, called from the main loop, ***execute for a reasonable amount of time***... allowing the loop to call the other functions at approximately the right frequency

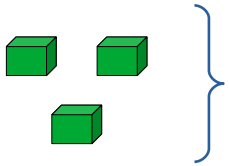
What we need is an execution model that supports kind of programming...

```
void _start() {
  uart_init(UART0);
  button_init_regs(BUTTON_BAR);
  led_init_regs(LED_BAR);
  for (;;) {
    button_and_led();
    echo();
  }
}
void button_and_led() {
  uint8_t status = button_get_status(BUTTON_BAR);
  if (status & 0x01)
    led_on(LED_BAR);
  else
    led_off(LED_BAR);
}
void echo() {
  // read available byte from the keyboard
  uint8_t code = uart_receive(UART0);
  if (code==0) return;
  // echo the character to the terminal
  uart_send(UART0, code);
  if (code == '\n') {
      shell(line, offset);
      offset=0;
    } else
      line[offset++]=(char)code;
}
```

© Pr. Olivier Gruber

Execution Models – Two Schools of thought

Several Tasks To Do...

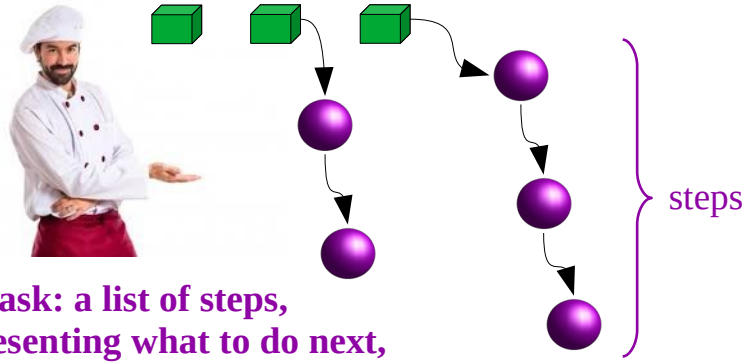


Two execution models are possible

Cook = Execution flow + Stack

One Cook = One Task
Multiple cooks → multiple tasks

One Cook = Multiple Tasks

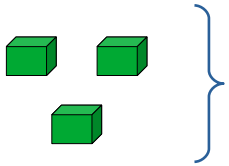


Per task: a list of steps, representing what to do next, in FIFO order per task.

© Pr. Olivier Gruber

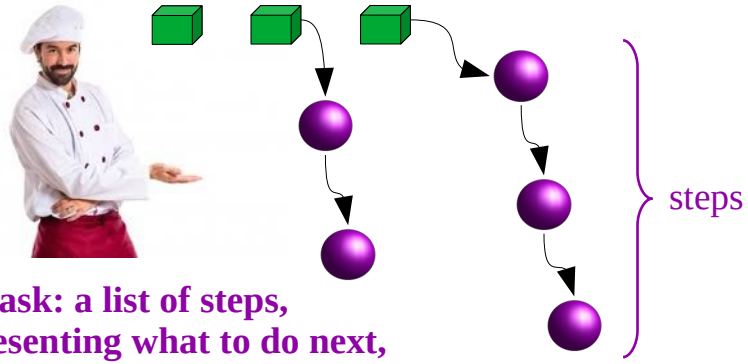
Execution Model – Reified Tasks or Not?

Several Tasks To Do...



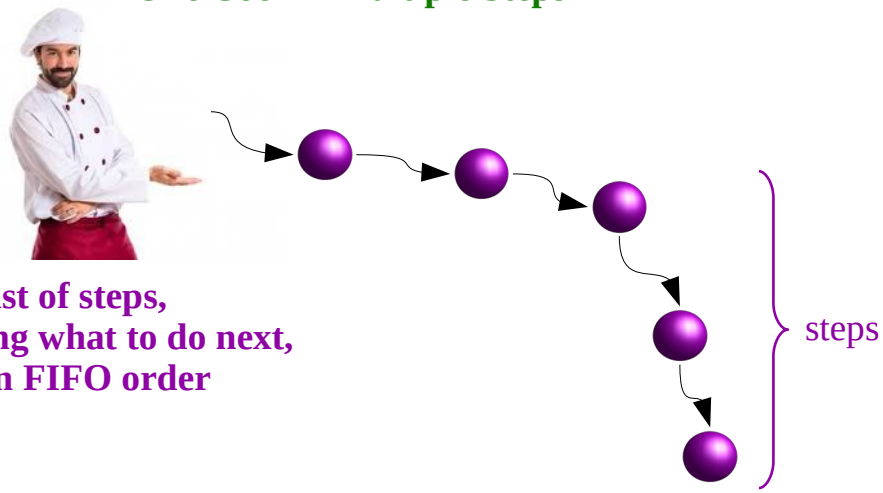
Cook = Execution flow + Stack

One Cook = Multiple Tasks



Per task: a list of steps, representing what to do next, managed in FIFO order

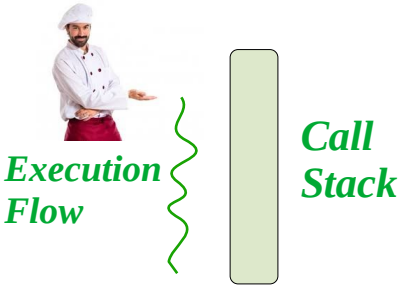
One Cook = Multiple Steps



Only one list of steps, representing what to do next, managed in FIFO order

© Pr. Olivier Gruber

Execution Model – Steps with ETA

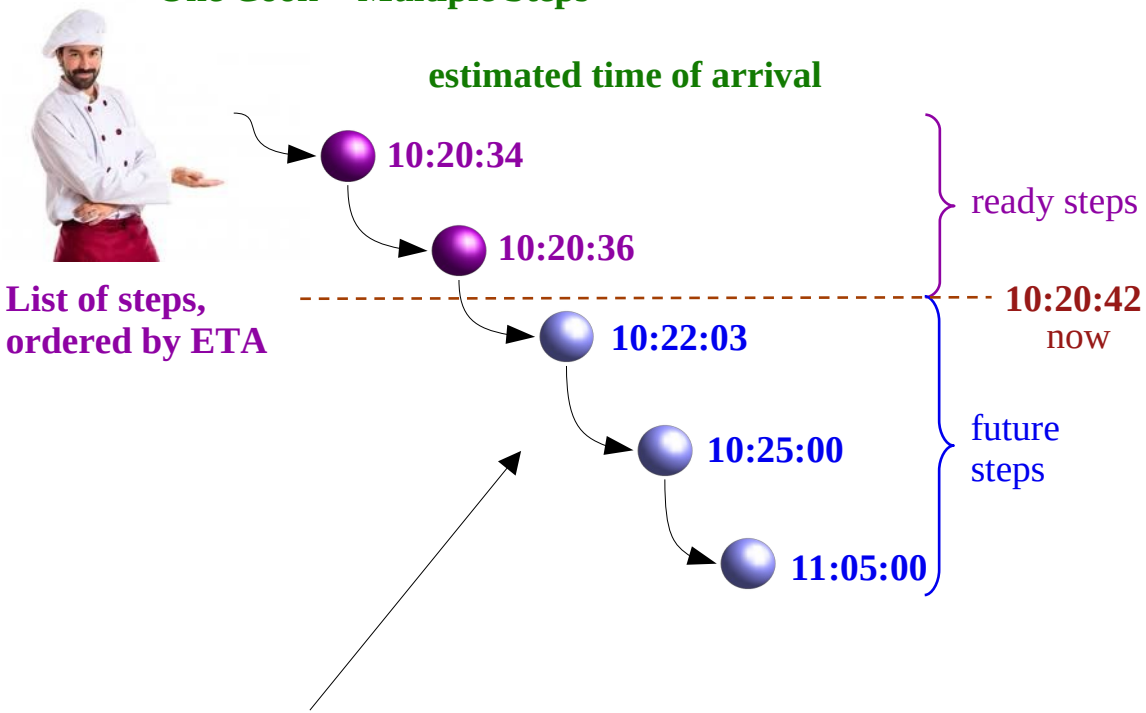


Cook = Execution flow + Stack

ETA: Estimated Time of Arrival

eta = now + delay

One Cook = Multiple Steps



Execution skew:

skew = now + first-event-eta

skew = 10:20:42 + 10:20:34

© Pr. Olivier Gruber

Event-oriented Programming – No Reified Tasks

- Event pump
  - One queue of events
  - Ordered by event ETA

```
// Post an new event on the FIFO queue
void event_post(void (*react)(void* cookie),
                void* cookie,
                uint32_t delay); // milliseconds
```

- Execution
  - Execution happens solely by executing a reaction
  - Each reaction runs to completion
  - Reactions must be "***short enough***"
  - ***This means no blocking, no waiting, no spinning***
  - Event reactions may post new reactions

```
struct event {
  void* cookie;
  void (*react)(void* cookie);
  uint64_t eta;
}


void _start() {

  ...
  for (;;) { // event pump,
    struct event* evt = event_pop();
    if (evt!=NULL)
      evt→react(evt→cookie);
    else
      sleep_until_next_event();
  }
}
```

- Event reactions
  - Function pointer with a cookie given back

© Pr. Olivier Gruber

Examples of Event-oriented Programming


```
void button_and_led_init() {
  event_post(button_and_led, NULL, 100);
}

void button_and_led(void* cookie) {
  uint8_t status = button_get_status(BUTTON_BAR);
  if (status & 0x01)
    led_on(LED_BAR);
  else
    led_off(LED_BAR);
  event_post(button_and_led, NULL, 100);
}
```

This is a clean and simple example of a periodic processing...

```
// Post an new event on the FIFO queue
void event_post(void (*react)(void* cookie),
                void* cookie,
                uint32_t delay); // milliseconds
```

```
struct event {
  void* cookie;
  void (*react)(void* cookie);
  uint64_t eta; // Estimated Time of Arrival
}


void _start() {  // event pump,
  button_and_led_init();
  ...
  for (;;) {
    struct event* evt = event_pop();
    if (evt!=NULL)
      evt→react(evt→cookie);
    else
      sleep_until_next_event();
  }
}
```

© Pr. Olivier Gruber

Examples of Event-oriented Programming


```
void echo_init() {
  event_post(echo, NULL, 1);
}

static void echo(void* cookie) {
  // read available byte from the keyboard
  uint8_t code = uart_receive(UART0);
  if (code==0) return;
  // echo the character to the terminal
  uart_send(UART0, code);
  if (code != '\n') {
    line[offset++]=(char)code;
    event_post(echo, NULL, 1);
    return;
  }
  shell(line, offset);
}

void echo_shell_done() {
  offset = 0;
  event_post(echo, NULL, 0);
}
```

```
// Post an new event on the FIFO queue
void event_post(void (*react)(void* cookie),
                void* cookie,
                uint32_t delay); // milliseconds
```

```
struct event {
  void* cookie;
  void (*react)(void* cookie);
  uint64_t eta; // Estimated Time of Arrival
}


void _start() {  // event pump,
  echo_init();
  ...
  for (;;) {
    struct event* evt = event_pop();
    if (evt!=NULL)
      evt→react(evt→cookie);
    else
      sleep_until_next_event();
  }
}
```

Nota Bene: the shell ***must*** execute the command as a sequence of short event reactions...

When done, it call "echo_shell_done()"

© Pr. Olivier Gruber

Event-oriented Programming – Analysis


```
void button_and_led_init() {
  event_post(button_and_led, NULL, 100);
}
void button_and_led(void* cookie) {
  uint8_t status = button_get_status(BUTTON_BAR);
  if (status & 0x01)
    led_on(LED_BAR);
  else
    led_off(LED_BAR);
  event_post(button_and_led, NULL, 100);
}
```

Is it possible to loose typed characters?

Can the button still feel unresponsive?

```
void echo_init() {
  event_post(echo, NULL, 1);
}
static void echo(void* cookie) {
  // read available byte from the keyboard
  uint8_t code = uart_receive(UART0);

  if (code==0) return;

  // echo the character to the terminal
  uart_send(UART0, code);

  if (code != '\n') {
    line[offset++]=(char)code;
    event_post(echo, NULL, 1);
    return;
  }
  shell(line, offset);
}
void echo_shell_done() {
  offset = 0;
  event_post(echo, NULL, 0);
}
```

```
struct event {
  void* cookie;
  void (*react)(void* cookie);
  uint64_t eta; // Estimated Time of Arrival
};


void _start() {  // event pump,
  button_and_led_init();
  echo_init();
  ...
  for (;;) {
    struct event* evt = event_pop();
    if (evt!=NULL)
      evt→react(evt→cookie);
    else
      sleep_until_next_event();
  }
}
```

© Pr. Olivier Gruber

Event-oriented Programming – Analysis


```
void button_and_led_init() {
  event_post(button_and_led, NULL, 100);
}
void button_and_led(void* cookie) {
  uint8_t status = button_get_status(BUTTON_BAR);
  if (status & 0x01)
    led_on(LED_BAR);
  else
    led_off(LED_BAR);
  event_post(button_and_led, NULL, 100);
}
```

```
void echo_init() {
  event_post(echo, NULL, 1);
}
static void echo(void* cookie) {
  // read available byte from the keyboard
  uint8_t code = uart_receive(UART0);

  if (code==0) return;

  // echo the character to the terminal
  uart_send(UART0, code);

  if (code != '\n') {
    line[offset++]=(char)code;
    event_post(echo, NULL, 1);
    return;
  }
  shell(line, offset);
}
void echo_shell_done() {
  offset = 0;
  event_post(echo, NULL, 0);
}
```

Is it possible to loose typed characters?

Can the button still feel unresponsive?

It all depends on how the shell is programmed...

If the shell uses short-enough reactions, everything is fine.

But what is short enough?

Less than the worst acceptable skew for the button, so probably like 50ms, half the button polling period at 100ms.

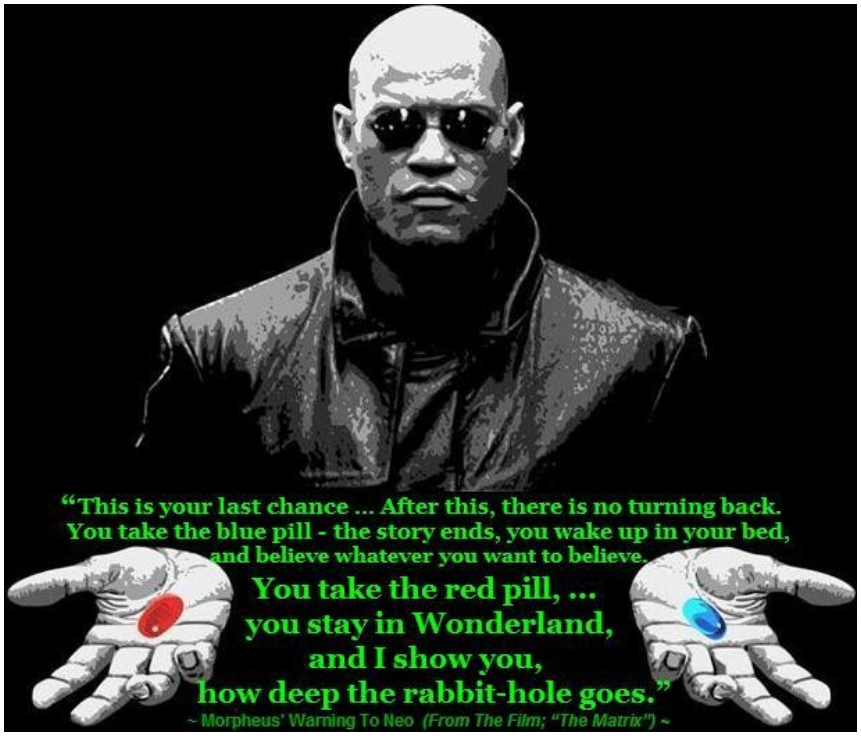
Dropping typed characters is unlikely because end users type in a command and wait for the result.

If we expect expert end user to type non-stop a stream of commands, then we would need to add some buffering to reading the UART0.

© Pr. Olivier Gruber

Conclusion

- ***Warning…***
  - Event-oriented programming is different
  - It will take time to master
  - Practice makes perfect



© Pr. Olivier Gruber